Lycée Duhamel du Monceau. Niveau : Première.		
	Numérique et Sciences Informatiques	
Listes, tuples et dictionnaires.		

I) LES LISTES PYTHON :

Les **listes** (ou **list** / **array**) en **python** sont une variable dans laquelle on peut mettre plusieurs variables.

Créer une liste en python

Pour créer une **liste**, rien de plus simple :

```
>>> liste = []
```

Vous pouvez voir le contenu de la **liste** en l'appelant comme ceci :

```
>>> liste
<type 'list'>
```

Ajouter une valeur à une liste python

Vous pouvez ajouter les valeurs que vous voulez lors de la création de la **liste python** :

```
>>> liste = [1,2,3]
>>> liste
[1, 2, 3]
```

Ou les ajouter après la création de la liste avec la méthode append (qui signifie "ajouter" en anglais) :

```
>>> liste = []
>>> liste
[]
>>> liste.append(1)
>>> liste
[1]
>>> liste.append("ok")
>>> liste
[1, 'ok']
```

On voit qu'il est possible de mélanger dans une même liste des variables de type différent. On peut d'ailleurs mettre une liste dans une liste.

Afficher un item d'une liste

Pour lire une liste, on peut demander à voir l'index de la valeur qui nous intéresse :

```
>>> liste = ["a","d","m"]
>>> liste[0]
'a'
>>> liste[2]
'm'
```

Le premier item commence toujours avec l'index 0. Pour lire le premier item on utilise la valeur 0, le deuxième on utilise la valeur 1, etc.

Il est d'ailleurs possible de modifier une valeur avec son index

```
>>> liste = ["a","d","m"]
>>> liste[0]
'a'
>>> liste[2]
'm'
>>> liste[2] = "z"
>>> liste
['a', 'd', 'z']
```

Supprimer une entrée avec un index

Il est parfois nécessaire de supprimer une entrée de la liste. Pour cela vous pouvez utiliser la fonction del.

```
>>> liste = ["a", "b", "c"]
>>> del liste[1]
>>> liste
['a', 'c']
```

Supprimer une entrée avec sa valeur
Il est possible de supprimer une entrée d'une liste avec sa valeur avec la méthode remove.

```
>>> liste = ["a", "b", "c"]
>>> liste.remove("a")
>>> liste
['b', 'c']
```

Inverser les valeurs d'une liste
Vous pouvez inverser les items d'une liste avec la méthode reverse.

```
>>> liste = ["a", "b", "c"]
>>> liste.reverse()
>>> liste
['c', 'b', 'a']
```

On peut vouloir **ordonner** une liste de nombres dans **l'ordre croissant**.

```
>>> a = [9, 7, 6, 9]
>>> a.sort()
>>> a
[6, 7, 9, 9]
```

Ou par ordre **décroissant**.

```
>>> a = [11, 7, 6, 9]
>>> a.sort(reverse = True)
>>> a
[11, 9, 7, 6]
```

Compter le nombre d'items d'une liste
Il est possible de compter le nombre d'items d'une liste avec la fonction len.

```
>>> liste = [1,2,3,5,10]
>>> len(liste)
5
```

Compter le nombre d'occurrences d'une valeur
Pour connaitre le nombre d'occurrences d'une valeur dans une liste, vous pouvez utiliser la méthode count.

```
>>> liste = ["a","a","a","b","c","c"]
>>> liste.count("a")
3
>>> liste.count("c")
2
```

Trouver l'index d'une valeur
La méthode index vous permet de connaitre la position de l'item cherché.

```
>>> liste = ["a","a","a","b","c","c"]
>>> liste.index("b")
3
```

Manipuler une liste
Voici quelques astuces pour manipuler des listes :

```
>>> liste = [1, 10, 100, 250, 500]
>>> liste[0]
1
>>> liste[-1] # Cherche la dernière occurrence
```

```
500
>>> liste[-4:] # Affiche les 4 dernières occurrences
[500, 250, 100, 10]
>>> liste[:] # Affiche toutes les occurrences
[1, 10, 100, 250, 500]
>>> liste[2:4] = [69, 70]
[1, 10, 69, 70, 500]
>>> liste[:] = [] # vide la liste
[]
```

Boucler sur une liste

Pour afficher les valeurs d'une liste, on peut utiliser une boucle :

```
>>> liste = ["a","d","m"]
>>> for lettre in liste:
...         print (lettre)
...
a
d
m
```

Si vous voulez en plus récupérer l'index, vous pouvez utiliser la fonction enumerate.

```
>>> for lettre in enumerate(liste):
...         print (lettre)
...
(0, 'a')
(1, 'd')
(2, 'm')
```

Les valeurs retournées par la boucle sont des **tuples.**

Copier une liste

Beaucoup de débutants font l'erreur de copier une liste de cette manière

```
>>> x = [1,2,3]
>>> y = x
```

Or si vous changez une valeur de la liste **y**, la liste **x** sera elle aussi affectée par cette modification :

```
>>> x = [1,2,3]
>>> y = x
>>> y[0] = 4
>>> x
[4, 2, 3]
```

En fait cette syntaxe permet de travailler sur un même élément nommé différemment

Alors comment copier une liste qui sera indépendante ?

```
>>> x = [1,2,3]
>>> y = x[:]
>>> y[0] = 9
>>> x
[1, 2, 3]
>>> y
[9, 2, 3]
```

Pour des données plus complexes, vous pouvez utiliser la fonction deepcopy du module copy

```
>>> import copy
>>> x = [[1,2], 2]
>>> y = copy.deepcopy(x)
>>> y[1] = [1,2,3]
>>> x
[[1, 2], 2]
>>> y
[[1, 2], [1, 2, 3]]
```

Transformer une string en liste

Parfois il peut être utile de transformer une chaine de caractère en liste. Cela est possible avec la méthode split.

```
>>> ma_chaine = "Olivier:ENGEL:Strasbourg"
>>> ma_chaine.split(":")
['Olivier', 'ENGEL', 'Strasbourg']
```

Transformer une liste en string

L'inverse est possible avec la méthode "**join**".

```
>>> liste = ["Olivier","ENGEL","Strasbourg"]
>>> ":".join(liste)
'Olivier:ENGEL:Strasbourg'
```

Trouver un item dans une liste

Pour savoir si un élément est dans une liste, vous pouvez utiliser le mot clé **in** de cette manière :

```
>>> liste = [1,2,3,5,10]
>>> 3 in liste
True
>>> 11 in liste
False
```

La fonction range

La fonction range génère une liste composée d'une simple suite arithmétique.

```
>>> a = range(5, 15, 2)
>>> list(a)
[5, 7, 9, 11, 13]
```

On a donc obtenu les entiers de 5 (inclus) à 15 (exclu) de 2 en 2 (on dit que l'incrément est 2). On peut ne pas préciser l'incrément et dans ce cas, l'incrément est 1.

Noter l'utilisation de la **fonction list()**, qui permet de faire afficher le contenu de la liste.

```
>>> B = range(3, 9)
>>> list(B)
[3, 4, 5, 6, 7, 8]
```

On peut en plus ne pas préciser l'entier de départ. Et dans ce cas, la valeur de départ est automatiquement 0.

```
>>> x = range(5)
>>> list(x)
[0, 1, 2, 3, 4]
```

Agrandir une liste par une liste

Pour mettre bout à bout deux listes, vous pouvez utiliser la méthode extend

```
>>> x = [1, 2, 3, 4]
>>> y = [4, 5, 1, 0]
>>> x.extend(y)
>>> print (x)
[1, 2, 3, 4, 4, 5, 1, 0]
```

Astuces

Afficher les 2 premiers éléments d'une liste

```
>>> liste = [1,2,3,4,5]
>>> liste[:2]
[1, 2]
```

Afficher le dernier item d'une liste :

```
>>> liste = [1, 2, 3, 4, 5, 6]
>>> liste[-1]
6
```

Afficher le 3ème élément en partant de la fin :

```
>>> liste = [1, 2, 3, 4, 5, 6]
>>> liste[-3]
4
```

Afficher les 3 derniers éléments d'une liste :

```
>>> liste = [1, 2, 3, 4, 5, 6]
>>> liste[-3:]
[4, 5, 6]
```

Vous pouvez additionner deux listes pour les combiner ensemble en utilisant l'opérateur + :

```
>>> x = [1, 2, 3]
>>> y = [4, 5, 6]
>>> x + y
[1, 2, 3, 4, 5, 6]
```

Vous pouvez même multiplier une liste :

```
>>> x = [1, 2]
>>> x*5
[1, 2, 1, 2, 1, 2, 1, 2, 1, 2]
```

Ce qui peut être utile pour initialiser une liste :

```
>>> [0] * 5
[0, 0, 0, 0, 0]
```

LES LISTES EN COMPREHENSION (OU COMPREHENSION DE LISTE) :

La compréhension de liste est une expression qui permet de construire une liste à partir de tout autre type itérable (liste, tuple, chaîne de caractères…). Le résultat obtenu est toujours une liste.

Une compréhension de liste toute simple

Un exemple valant mieux qu'un long discours, nous allons créer une liste de nombres entiers :
liste_initiale = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
À l'aide d'une boucle **for** et de la méthode **append()**, nous allons ajouter à une nouvelle liste baptisée **liste_2**, chaque élément de **liste_initiale** multiplié par deux.

```
liste_initiale = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
liste_2 = []
for n in liste_initiale:
    liste_2.append(n*2)
print(liste_2)
```

Résultat : liste_2 = [0, 2, 4, 6, 8, 10 ,12, 14, 16, 18, 20, 22, 24, 26, 28, 30]
On a donc utilisé une boucle **for** pour écrire ce petit bout de code de cinq lignes. Mais on aurait tout aussi bien pu utiliser une compréhension de liste pour obtenir le même résultat. Voici la syntaxe de cette dernière :

```
liste_initiale = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
liste_2 = [n*2 for n in liste_initiale]
print(liste_2)
```

Résultat : liste_2 = [0, 2, 4, 6, 8, 10 ,12, 14, 16, 18, 20, 22, 24, 26, 28, 30]
Que remarque-t-on ? Le code est plus concis puisqu'il tient sur trois lignes au lieu de cinq.

Exemples simples :

```
>>> l = [x for x in range(0, 10)]
>>> l
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> l = [2*x for x in range(0, 10)]
>>> l
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

Une compréhension de liste avec une condition

On souhaite créer une nouvelle liste de nombres pairs multipliés par 2 à partir de **liste_initiale** qui contient des nombres pairs et impairs. Il va donc falloir introduire une condition pour ignorer les nombres impairs. Comparons une nouvelle fois le code écrit avec une boucle **for** et celui écrit avec une compréhension de liste.
Avec une boucle for

```
liste_initiale = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
liste_pair = []
for n in liste_initiale:
    if n % 2 == 0:
```

```
    liste_pair.append(n*2)
print(liste_pair)
```

Résultat : liste_pair = [0, 4, 8, 12, 16, 20, 24, 28]

Avec une compréhension de liste

On introduit la condition à la fin de la compréhension de liste. Au lieu d'avoir six lignes de code, on en a plus que trois.

```
liste_initiale = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
liste_pair = [n*2 for n in liste_initiale if n % 2 == 0]
print(liste_pair)
```

Résultat : liste_pair = [0, 4, 8, 12, 16, 20, 24, 28]

Une compréhension de liste avec une expression conditionnelle

On souhaite créer une nouvelle liste avec les éléments de **liste_initiale**. Les chiffres pairs seront multipliés par deux, les chiffres impairs par trois. Il va donc falloir introduire une condition **if… else**.

Avec une boucle for

```
liste_initiale = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
nouvelle_liste = []
for n in liste_initiale:
  if n % 2 == 0:
    nouvelle_liste.append(n*2)
  else:
    nouvelle_liste.append(n*3)
print(nouvelle_liste)
```

Résultat : nouvelle_liste = [0, 3, 4, 9, 8, 15, 12, 21, 16, 27, 20, 33, 24, 39, 28, 45]

Avec une compréhension de liste

Ça se complique un peu car il faut utiliser une expression conditionnelle qui évalue si la condition est vraie (**True**) ou fausse (**False**). Mais le résultat est au rendez-vous puisqu'au lieu d'avoir huit lignes de code, on n'en a que trois !

```
liste_initiale = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
nouvelle_liste = [n*2 if n % 2 == 0 else n*3 for n in liste_initiale]
print(nouvelle_liste)
```

Une compréhension de liste avec une liste de listes

Il est possible de faire une compréhension de liste avec une liste de listes. Pour s'y retrouver dans l'imbrication des instructions, le plus simple est de comparer la syntaxe d'une boucle **for** avec celle d'une compréhension de liste.

Avec une boucle for

```
liste_initiale = [[0, 'a'], [2, 'b'], [3, 'c']]
nouvelle_liste = []
for i in liste_initiale :
  for n in i :
    nouvelle_liste.append(n*2)
print(nouvelle_liste)
```

Résultat: nouvelle_liste = [0, 'aa', 4, 'bb', 6, 'cc']

Avec une compréhension de liste

```
liste_initiale = [[0, 'a'], [2, 'b'], [3, 'c']]
nouvelle_liste = [n*2 for i in liste_initiale for n in i]
print(nouvelle_liste)
```

Résultat: nouvelle_liste = [0, 'aa', 4, 'bb', 6, 'cc']

Avec une expression conditionnelle

Par exemple, je veux multiplier les nombres entiers par deux et les strings par trois.

```
liste_initiale = [[0, 'a'], [2, 'b'], [3, 'c']]
nouvelle_liste = [n*2 if type(n) == int else n*3 for i in liste_initiale for n in i]
print(nouvelle_liste)
```

Résultat: [0, 'aaa', 4, 'bbb', 6, 'ccc']

Dans ce dernier exemple, si on avait fait usage d'une boucle **for**, on aurait écrit neuf lignes de code au lieu de trois !

```
liste_initiale = [[0, 'a'], [2, 'b'], [3, 'c']]
nouvelle_liste = []
for i in liste_initiale:
  for n in i:
    if type(n) == int:
```

```
        nouvelle_liste.append(n*2)
    else:
        nouvelle_liste.append(n*3)
print(nouvelle_liste)
```

Cela dit, ce n'est pas non plus le concours de celui qui écrira le plus d'instructions sur la même ligne de code. Il faut que l code reste déchiffrable.

Les compréhensions de liste avec les autres types itérables

Les compréhensions de liste fonctionnent avec les autres types itérables mais le résultat obtenu est toujours une liste. Par exemple, avec une chaîne de caractères :

```
prenom = "Gustave"
liste_lettres = [lettre for lettre in prenom]
print(liste_lettres)
```

Résultat: ['G', 'u', 's', 't', 'a', 'v', 'e']

Conclusion

La compréhension de liste est une méthode puissante qui bien utilisée, rend le code plus concis, plus élégant et plus facile à comprendre. Voici la page consacrée aux compréhensions de liste dans la documentation officielle du langage Python.
Pour les personnes intéressées, il existe également une compréhension de dictionnaire et d'ensemble.

II) LES TUPLES PYTHON :

Un **tuple** est une **liste** qui ne peut plus être modifiée.

Créer un tuple
Pour créer un **tuple**, vous pouvez utiliser la syntaxe suivante :

```
>>> mon_tuple = ()
```

Ajouter une valeur à un tuple
Pour créer un **tuple** avec des valeurs, vous pouvez le faire de cette façon :

```
>>> mon_tuple = (1, "ok", "olivier")
```

Les parenthèses ne sont pas obligatoires mais facilite la lisibilité du code (rappelons que la force de python est sa simplicité de lecture) :

```
>>> mon_tuple = 1, 2, 3
>>> type(mon_tuple)
<type 'tuple'>
```

Lorsque vous créez un tuple avec une seule valeur, n'oubliez pas d'y ajouter une virgule, sinon ce n'est pas un tuple.

```
>>> mon_tuple = ("ok")
>>> type(mon_tuple)
<type 'str'>
>>> mon_tuple = ("ok",)
>>> type(mon_tuple)
<type 'tuple'>
```

Afficher une valeur d'un tuple
Le tuple est une sorte de liste, on peut donc utiliser la même syntaxe pour lire les données du tuple.

```
>>> mon_tuple[0]
1
```

Et évidemment si on essaie de changer la valeur d'un index, l'interpréteur nous insulte copieusement :

```
>>> mon_tuple[1] = "ok"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

A quoi sert un tuple alors ?
Le tuple permet une affectation multiple :

```
>>> v1, v2 = 11, 22
>>> v1
11
>>> v2
22
```

Il permet également de renvoyer plusieurs valeurs lors d'un appel d'une fonction :

```
>>> def donne_moi_ton_nom():
...         return ("olivier", "engel")
...
>>> donne_moi_ton_nom()
('olivier', 'engel')
```

On utilisera un tuple pour définir des sortes de constantes qui n'ont donc pas vocation à changer.

III) <u>LES DICTIONNAIRES PYTHON</u> :

Un **dictionnaire** en **python** est une sorte de **liste** mais au lieu d'utiliser des **index**, on utilise des **clés**, c'est à dire des valeurs autres que numériques.

Comment créer un dictionnaire ?
Pour initialiser un **dictionnaire**, on utile la syntaxe suivante :

```
>>> a = {}
```

Comment ajouter des valeurs dans un dictionnaire ?
Pour ajouter des valeurs à un dictionnaire il faut indiquer une clé ainsi qu'une valeur :

```
>>> a = {}
>>> a["nom"] = "engel"
>>> a["prenom"] = "olivier"
>>> a
{'nom': 'engel', 'prenom': 'olivier'}
```

Vous pouvez utiliser des clés numériques comme dans la logique des **listes**.

Récupérer une valeur dans un dictionnaire
La méthode get vous permet de récupérer une valeur dans un dictionnaire et si la clé est introuvable, vous pouvez donner une valeur à retourner par défaut :

```
>>> data = {"name": "Olivier", "age": 30}
>>> data.get("name")
'Olivier'
>>> data.get("adresse", "Adresse inconnue")
'Adresse inconnue'
```

Vérifier la présence d'une clé dans un dictionnaire
Vous pouvez utiliser la méthode suivante pour vérifier la présence d'une clé que vous cherchez :

```
>>> "nom" in a
True
```

Supprimer une entrée de dictionnaire
Il est possible de supprimer une entrée en indiquant sa clé, comme pour les listes :

```
>>> del a["nom"]
>>> a
{'prenom': 'olivier'}
```

Récupérer les clés par une boucle
Pour récupérer les clés on utilise la méthode ***keys***

```
>>> fiche = {"nom":"engel","prenom":"olivier"}
>>> for cle in fiche.keys():
...         print (cle)
...
nom
```

```
prenom
```

Récupérer les valeurs par une boucle
Pour cela on utilise la méthode ***values***

```
>>> fiche = {"nom":"engel","prenom":"olivier"}
>>> for valeur in fiche.values():
...         print (valeur)
...
engel
olivier
```

Récupérer les clés et les valeurs par une boucle
Pour récupérer les clés et les valeurs en même temps, on utilise la méthode ***items*** qui retourne un tuple.

```
>>> fiche = {"nom":"engel","prenom":"olivier"}
>>> for cle,valeur in fiche.items():
...         print (cle, valeur)
...
nom engel
prenom olivier
```

Utiliser des tuples comme clé
Une des forces de python est la combinaison tuple/dictionnaire qui fait des merveilles dans certains cas comme lors de l'utilisation de coordonnées.

```
>>> b = {}
>>> b[(3,2)]=12
>>> b[(4,5)]=13
>>> b
{(4, 5): 13, (3, 2): 12}
```

Créer une copie indépendante d'un dictionnaire
Comme pour toute variable, vous ne pouvez pas copier un dictionnaire en faisant **dic1 = dic2** :

```
>>> d = {"k1":"olivier", "k2":"engel"}
>>> e = d
>>> d["k1"] = "XXX"
>>> e
{'k2': 'engel', 'k1': 'XXX'}
```

Pour créer une copie indépendante vous pouvez utiliser la méthode copy :

```
>>> d = {"k1":"olivier", "k2":"engel"}
>>> e = d.copy()
>>> d["k1"] = "XXX"
>>> e
{'k2': 'engel', 'k1': 'olivier'}
```

IV) <u>APPLICATIONS</u> :

<u>En résumé</u> :
Une chaîne est une suite éventuellement vide non modifiable de caractères :

```
chaine = 'isbs Paris'
```

Une liste est une suite éventuellement vide modifiable de données (ici une liste de caractères) :

```
liste = ['i','s','b','s',' ','P','a','r','i','s']
```

Un tuple est une suite éventuellement vide non modifiable de données (ici un tuple de caractères) :

```
nuplet = ('i','s','b','s',' ','P','a','r','i','s')
```

Un ensemble est une collection éventuellement vide modifiable de données sans ordre défini et sans doublons (ici un ensemble de caractères) :

```
ensemble = set(('i','s','b',' ','P','a','r'))
```

Un dictionnaire est une collection éventuellement vide modifiable de couples (clé, valeur) sans ordre défini et sans doublons de clés (ici un dictionnaire de caractères associés à leurs fréquences dans la chaîne 'isbs Paris') :

```
dico = {'i':2,'s':3,'b':1,' ':1,'P':1,'a':1,'r':1}
```

Passer de l'un à l'autre de ces types de données, savoir quand les utiliser est une question d'expérience et de bons sens.
Regarder ce que donnent les conversions suivantes :

```
# Avec
liste=['i', 's', 'b', 's', ' ', 'P', 'a', 'r', 'i', 's']
# de type list
list(liste) # --> ['i', 's', 'b', 's', ' ', 'P', 'a', 'r', 'i', 's']
tuple(liste) # --> ('i', 's', 'b', 's', ' ', 'P', 'a', 'r', 'i', 's')
str(liste) # --> "['i', 's', 'b', 's', ' ', 'P', 'a', 'r', 'i', 's']"
dict((x,0) for x in liste) # --> {'a': 0, ' ': 0, 'b': 0, 'i': 0, 'P': 0, 's': 0, 'r': 0}
set(liste) # --> {'a', ' ', 'b', 'i', 'P', 's', 'r'}
# Avec
chaine='isbs Paris'
# de type str
list(chaine) # --> ['i', 's', 'b', 's', ' ', 'P', 'a', 'r', 'i', 's']
tuple(chaine) # --> ('i', 's', 'b', 's', ' ', 'P', 'a', 'r', 'i', 's')
str(chaine) # --> 'isbs Paris'
dict((x,0) for x in chaine) # --> {'a': 0, ' ': 0, 'b': 0, 'i': 0, 'P': 0, 's': 0, 'r': 0}
set(chaine) # --> {'a', ' ', 'b', 'i', 'P', 's', 'r'}
# Avec
ensemble={'a', ' ', 'b', 'i', 'P', 's', 'r'}
# de type set
list(ensemble) # --> ['a', ' ', 'b', 'i', 'P', 's', 'r']
tuple(ensemble) # --> ('a', ' ', 'b', 'i', 'P', 's', 'r')
str(ensemble) # --> "{'a', ' ', 'b', 'i', 'P', 's', 'r'}"
dict((x,0) for x in ensemble) # --> {'a': 0, ' ': 0, 'b': 0, 'i': 0, 'P': 0, 's': 0, 'r': 0}
set(ensemble) # --> {'a', ' ', 's', 'b', 'P', 'i', 'r'}
#Avec
nuplet=('i', 's', 'b', 's', ' ', 'P', 'a', 'r', 'i', 's')
# de type tuple
list(nuplet) # --> ['i', 's', 'b', 's', ' ', 'P', 'a', 'r', 'i', 's']
tuple(nuplet) # --> ('i', 's', 'b', 's', ' ', 'P', 'a', 'r', 'i', 's')
str(nuplet) # --> "('i', 's', 'b', 's', ' ', 'P', 'a', 'r', 'i', 's')"
dict((x,0) for x in nuplet) # --> {'a': 0, ' ': 0, 'b': 0, 'i': 0, 'P': 0, 's': 0, 'r': 0}
set(nuplet) # --> {'a', ' ', 'b', 'i', 'P', 's', 'r'}
```

Exécutez-le et comprenez les affichages.

Exercice 1 : Les nombres de Armstrong
Un nombre est dit de Armstrong s'il est égal à la somme des cubes de ses chiffres (par exemple, $371 = 3^3 + 7^3 + 1^3$).
Générez la liste des nombres de Armstrong inférieurs à 1000.

Exercice 2 : Génération de bigrammes
Générez la liste de tous les bigrammes (chaîne de deux lettres) possibles avec l'alphabet français.
Voici par exemple la solution avec la compréhension de liste :

```
>>> [chr(ord('A')+x)+chr(ord('A')+y) for y in range(26) for x in range(26)]
['AA', 'BA', 'CA', 'DA', 'EA', 'FA', 'GA', 'HA', 'IA', 'JA', 'KA',
...
'VZ', 'WZ', 'XZ', 'YZ', 'ZZ']
```

Donner la solution sans compréhension de liste (avec deux boucles imbriquées).

Exercice 3 : Fréquences de caractères
Soit la chaîne de caractères :

```
>>> p = "portez ce vieux whisky au juge blond qui fume"
```

- Créez le dictionnaire des fréquences de cette chaîne (avec l'association clef = caractère, valeur = nombre d'apparitions du caractère dans la chaîne). Utilisez dict, str.count, set.
- Créez la liste correspondante à ce dictionnaire. Utilisez les list en compréhension.

- Triez cette liste par ordre décroissant de fréquences. Utilisez list.sort(), ou sorted(), (chercher la différence entre les deux). Cette partie est plus délicate car il va falloir trier selon le deuxième élément des éléments de la liste (c'est-à-dire selon la fréquence), puisque la liste créée sera une liste de tuples.
Pour ceci on peut s'inspirer de ce qui suit :

Tri personnalisé avec key

Parfois on a besoin de trier sur quelque chose de plus complexe qu'une lettre ou un chiffre. Par exemple, vous avez des scores dans un dictionnaires. Un dictionnaire n'est pas ordonné. Si vous imprimez un classement, il faut en faire une liste de tuples :

```
>>> scores = {'Robert': 2, 'Roger': 1, 'Gertrude': 4, 'Cunegonde': 3}
>>> scores.items()
[('Gertrude', 4), ('Cunegonde', 3), ('Robert', 2), ('Roger', 1)]
```

Et il faut maintenant ordonner ça sur le deuxième élément de chaque tuple : le score. Seulement si on appelle sorted(), il va trier sur l'ordre naturel, donc il va commencer par le premier élément, ça ne marchera pas :

```
>>> sorted(scores.items())
[('Cunegonde', 3), ('Gertrude', 4), ('Robert', 2), ('Roger', 1)]
```

C'est là qu'intervient le paramètre key. key est très particulier, c'est un paramètre qui attend qu'on lui passe un callback, donc key attend qu'on lui passe **une fonction**. Pas le résultat d'une fonction. La fonction elle-même.
Cette fonction doit attendre en paramètre un élément, et en **extraire la partie sur laquelle on veut trier**.
Par exemple dans le cas des scores, on doit passer à key une fonction qui attend un tuple en paramètre, et retourne le score :

```
>>> def get_score(nom_et_score):
...     return nom_et_score[1] # retourne le 2nd element du tuple
...
>>> sorted(scores.items(), key = get_score) # on passe la fonction a key
[('Roger', 1), ('Robert', 2), ('Cunegonde', 3), ('Gertrude', 4)]
```

Et voilà, c'est trié dans le bon ordre car sorted() va utiliser get_score() pour récupérer la valeur de chaque élément un par un, sur laquelle on va trier la liste (selon l'ordre naturel de cette valeur).

On peut aussi utiliser une fonction lambda :

```
>>> sorted(scores.items(), key = lambda position : position[1])
```

On définit une fonction lambda dont le nom est position. position contiendra un élément de la liste (c'est-à-dire un tuple). Si on retourne position[1], cela signifie qu'on veut récupérer le score (en deuxième position). Souvenez-vous, on commence toujours à 0.

On peut utiliser key pour faire des choses beaucoup plus complexes, comme comparer sur des valeurs qui n'existent pas, mais que l'on calcule :

```
>>> def carre(val): # on va ordonner par valeur de carré
...     return val * val
...
>>> sorted([-1, -2, 0, 3], key = carre)
[0, -1, -2, 3]
```

Ici, un carré est toujours positif, du coup on retrouve -1 et -2 devant 0, car leurs carrés 1 et 4 sont plus grands que le carré 0 de 0.
Ne vous embrouillez pas : la valeur retournée par carre() n'est pas mise dans la liste, elle est juste utilisée pour choisir l'ordre d'un élément dans la liste.

Exercice 4 : The Look and Say Sequence
Combien de chiffres comporte le n^e terme de la suite suivante :
1 11 21 1211 111221 312211 13112221 1113213211 31131211131221…
Avez-vous deviné la règle de construction de cette suite ? Le premier terme se compose de un « un », puis le deuxième énonce le premier « un un ». Ce deuxième terme se compose de deux « un », donc le troisième, énonçant le deuxième, est « deux un ». Il est composé de un « deux » et de un « un », et donc est suivi par « un deux un un », et ainsi de suite.
J.H. Conway, The Book of Numbers, Springer-Verlag, 1996
Si la suite de Conway commence par $u_0 = 1$ et $u_1 = 11$, quel est le nombre de chiffres de u_{26} ?

Exercice 5 : Un palindrome de 1 247 mots
Le site http://homepage.urbanet.ch/cruci.com/lexique/palindrome.htm nous donne le palindrome record de

Georges Perec.
Trace l'inégal palindrome ..... e mord ni la plage ni l'écart.
Georges Perec, La clôture et autre poèmes, Hachette, 1980
Téléchargez ce texte et écrivez les fonctions nécessaires à vérifier que c'est un palindrome
Note : Une erreur de transcription fait que le texte trouvé sur le site indiqué n'est pas un palindrome : trouvez la correction à faire dans le texte pour qu'il devienne un vrai palindrome.

Exercice 6 : Bingo des réunions
Vous avez des difficultés à supporter les réunions ?
L'ennui et le sommeil vous prennent pendant les conférences ?
Vos problèmes ont vécu. Car il existe maintenant le nouveau BINGO DES RÉUNIONS, une méthode très efficace pour garder la concentration lors des séances ou réunions.
Comment jouer ?
1. Faites un petit tableau comme l'exemple ci-dessous, avant la séance.
2. Chaque fois qu'un des mots présents dans les cases est prononcé, biffez-le.
3. Dès qu'une ligne, une colonne ou une diagonale est remplie, criez « BINGO ».

Potentialité	Enjeux	Ressources	Référentiel
Synergie	Cohérence	Relancer	Motiver
Partenaires	Décaler	Compétence	En fait
Acteur	Référencement	Signal	Adéquation

À l'aide de la liste suivante, que vous pouvez compléter, écrivez un générateur de grille de bingo des réunions

```
s=['Acteur', 'Activation', 'Adéquation', 'Agenda', 'Alambiqué', 'Attitude',
'Back Office', 'Blogosphère', 'Booster', 'Brainstorming', 'Budget',
'Business plan', 'Buzz', 'Bénéfice', 'Challenge', "Champ d'application",
'Cibler', 'Client', 'Cohérence', 'Communauté', 'Compétence', 'Convergence',
'Corrélation', 'Coeur de métier', 'Durable', 'Décaler', 'Efficacité',
'En fait', 'Enjeux', 'Externalisation', 'Faisabilité', 'Finaliser',
'Fondamentaux', 'Gagnant-gagnant', 'Gestion', 'Implémenter', 'Influence',
'Interface', 'Investissement', 'Leadership', 'Management', 'Mentalité',
'Mise en oeuvre', 'Modèle', 'Motiver', 'Mutualiser', 'Méthodologie',
'Nuage (cloud)', 'Optimiser', 'Partenaires', 'Périmètre','Phase',
'Point de vue', 'Positiver', 'Potentialité', 'Problématique', 'Process',
'Projet', 'Qualité', 'Recentrer', 'Référentiel','Relancer', 'Rendement',
'Ressources', 'Réduction', 'Réformer', 'Référencement', 'Référentiel',
'Réseaux sociaux', 'Ressources','Résultats', 'Signal', 'Significativité',
'Solutionner', 'Stratégie', 'Synergie', 'Team', 'Transversal', 'Typologie',
'Unicité', 'Veille technologique']
```

Exercice 7 : Échanger les clés et les valeurs d'un dictionnaire
Écrire une fonction Python echange(Dict) qui échange les clés et les valeurs d'un dictionnaire Dict (ce qui permettra par exemple de transformer un dictionnaire anglais/français en un dictionnaire français/anglais). On suppose que le dictionnaire ne contient pas plusieurs valeurs identiques.
Exemple d'exécution :

```
>>>Article={ 'Référence': 31654 , 'désignation': 'Souris' , 'prix': 30.25}
>>>Article=echange(Article)
>>>Article
{'Souris': 'désignation', 31654: 'Référence', 30.25: 'prix'}
```

Exercice 8 : Dictionnaire des fréquences d'une chaine
Dictionnaire des fréquences d'une chaine : Script en Python qui permet de Créer le dictionnaire des fréquences d'une chaine, saisie par l'utilisateur, (avec l'association clef = caractère, valeur = nombre d'apparitions du caractère dans la chaîne) puis qui il afficher le dictionnaire par ordre croissant de fréquences.

Pour cela :
- Créer une fonction count_car(ch,c) qui retourne le nombre d'apparitions du caractère c dans la chaîne ch.
- Créer une fonction Dictionnaire_frequence(chaine) qui retourne le dictionnaire des fréquences d'une chaine
- Créer une fonction affichage_frequence(chaine) qui afficher le dictionnaire par ordre croissant de fréquences

Exemple d’exécution :

```
Entrer une chaine de caractère : lepython_point_com
h   est apparue 1 fois
m  est apparue 1 fois
l    est apparue 1 fois
i    est apparue 1 fois
e   est apparue 1 fois
c   est apparue 1 fois
y   est apparue 1 fois
t    est apparue 2 fois
_   est apparue 2 fois
n   est apparue 2 fois
p   est apparue 2 fois
o  est apparue 3 fois
```